Milestone 5 — Visualization & Dashboard Development

Project: AI_PrognosAI — Remaining Useful Life (RUL) Prediction

Date: October 24, 2025

Author: Manju Shree

1. Objective
Create interactive visualizations and a dashboard to present RUL trends, predictions, and alerts. Deliverables include interactive charts displaying RUL trends over time, dashboards showing current RUL predictions and alert zones, and a user-friendly interface (Streamlit).

2. Summary of Implementation
The provided Streamlit application implements an interactive Prognostics Dashboard with:
- Model and scaler loading (saved during Milestone 4).
- Preprocessed test data loading (NPZ format).
- Scaling of sequence windows and model prediction.
- Alignment of sequence-level predictions with true RUL values.
- Alert level logic (Normal / Warning / Critical) based on predicted RUL thresholds.
- Interactive visualizations using Plotly:
- Line chart showing Actual vs Predicted RUL over time.
- Scatter chart of Predicted RUL with alert color mapping.
- Recent alerts table (last 20 entries) with conditional coloring.
- Metric summary (RMSE, MAE, R^2) displayed as dashboard KPIs.

3. Key Files & Paths (as used in the code)
- Model: models_m2/optimized_fd1_milestone4.h5
- Scaler: models_m2/scaler_fd1_milestone4.save
- Processed test data: processed/fd1_test_ws30.npz
- Streamlit app script: app_streamlit_prognosai.py (or similar)

4. Dashboard UX & Visual Components
- Top banner and title: quick context and project name.
- KPI cards: RMSE, MAE, R^2 for quick performance snapshot.
- RUL Trends: dual-line plot (Actual vs Predicted) to compare temporal behavior.
- Alerts scatter: colored by alert level to highlight risky periods.
- Recent Alerts table: displays latest entries with conditional coloring for quick triage.

5. Alert Thresholds & Logic
- Example thresholds used in the dashboard:
- Normal: Predicted RUL > 70
- Warning: 30 < Predicted RUL ≤ 70
- Critical: Predicted RUL ≤ 30
- Recommendations:
- Tune thresholds based on operational lead time and historical failure data.
- Require N consecutive low-RUL predictions before triggering persistent alerts to reduce false positives.
- Add confidence measures (ensembles or MC Dropout) to reduce noisy alerts.

6. Deployment & Running Instructions
1. Install dependencies:
pip install streamlit plotly joblib tensorflow scikit-learn pandas numpy
2. Ensure model, scaler and processed NPZ files are placed in the correct paths.
3. Run the Streamlit app:
streamlit run app_streamlit_prognosai.py
4. Open the provided local URL (e.g., http://localhost:8501) to view the dashboard.

7. Performance & Usability Notes
- For responsive UX, precompute predictions offline and serve them as static JSON/CSV if model inference is slow.
- Use caching (st.cache_data or st.cache_resource) in Streamlit to avoid reloading models on each interaction.
- For production, containerize the app (Docker) and deploy behind a lightweight web server (nginx) with autoscaling if needed.

8. Extensions & Next Steps
- Add per-engine drilldown: select engine_id to view sequence trajectories.
- Add time-series playback: animate predicted RUL as cycles progress.
- Integrate alert notification (email/SMS) for critical alerts.
- Add user authentication and role-based views for operators vs analysts.
- Expose an API endpoint to accept live telemetry and return alert decisions.

9. Appendix — Example Code Snippets
- Caching model load in Streamlit:
```python
@st.cache_resource
def load_model_cached(path):
return load_model(path, compile=False)
```
- Caching scaler:
```python
@st.cache_resource
def load_scaler(path):
return joblib.load(path)
```

End of Milestone 5 Report.